Does Sparse Training Make Neurons Easier to Read? Capacity and Monosemanticity in a Toy Model of Superposition

Narayan Srinivasa

Independent Researcher, physynapse@gmail.com

Abstract

Individual neurons in trained networks often respond to several unrelated features at once, which makes them hard to interpret. A leading explanation is superposition, in which a network represents more features than it has neurons by letting features share neurons. We ask whether training with sparse weights from the start makes neurons easier to read, and what it costs in capacity. We use a toy model of superposition with 128 known, sparse input features and a one-hidden-layer ReLU network that computes their absolute values. We compare dense training, always-sparse weight training by Guided Stochastic Exploration (GSE), top-k activation sparsity, their combination, and a control that uses the same kind of random sparse pattern as GSE but never changes it. When features are rare, sparse training makes neurons much easier to read. Removing one neuron of a dense network affects about 14 features, against about 5 for GSE at 50% sparsity, and GSE gives up little capacity to get there. In wider networks it gives up none, and its neurons stay clean across a tenfold range of learning rates, whereas those of the dense network do not. A fixed random mask gives still cleaner neurons but far less capacity, so rewiring mainly recovers capacity, and the advantage shrinks as features become denser. The original method ranks connections across all layers together, which removes most of the advantages of GSE in this network, so we rank within each layer's weight matrix instead. The benefit also depends on the route. Pruning a trained dense network keeps its capacity but not its clean neurons, and retraining on the pattern that GSE finds recovers much of its capacity but gives less clean neurons. When features come in families that appear together, dense neurons mix several families, and sparse training makes neurons serve one family. These results come from a synthetic setting with three to five seeds per condition, and they motivate testing sparse training in language models, where easier-to-read neurons would matter most.

Index Terms: interpretability, superposition, monosemanticity, sparse training, dynamic sparse training, toy models.

I. Introduction

Interpretability research tries to explain what each unit in a neural network does. Early work visualized the inputs that most strongly excite a unit [1], built interfaces that combine such visualizations with attribution to explain a network's behavior [2], and argued that features and the circuits that connect them are the right objects of study [3]. This approach meets a recurring obstacle: individual neurons often respond to several unrelated features, a property known as *polysemanticity*, so that reading a neuron does not reliably tell us what it means.

A widely discussed explanation of polysemanticity is superposition. When the features a network must represent are sparse, it can store more of them than it has neurons by letting several features share a neuron, accepting a small amount of interference in exchange for capacity [4]. A similar idea appears in neuroscience, where the cell-assembly concept holds that a single neuron can take part in many

assemblies and so in many contexts and meanings [5], [6]. For example, neurons in the prefrontal cortex have been found to respond to mixtures of task variables rather than to a single one [7]. In artificial networks, toy models with known ground-truth features show that the degree of superposition depends on the sparsity and importance of the features, and that a network can even compute on features stored in superposition [4]. Later work builds on this account. Scherlis et al. [8] treat a network's capacity as a limited budget shared among its features and show that polysemantic neurons arise when the budget must cover more features than there are neurons. Henighan et al. [9] show that superposition also shapes how a small network stores individual training examples. On this view, polysemanticity is not a defect of training but a consequence of fitting many features into few neurons. The account is now disputed for real networks. A recent essay argues that the canonical examples of superposition were misread, that most of the remaining polysemanticity is recursive generalization, in which a neuron's meaning grows outward step by step into related territory, and that superposition, in which unrelated features share a neuron, should be far down the list of explanations [10].

A common response to superposition is to stop reading neurons directly and learn a better basis instead. Sparse autoencoders trained on a model's activations recover directions that are more interpretable than individual neurons, first in a small transformer [11], then in language models [12], and later in large production models [13]. Variants that keep only the *K* largest latent activations make the sparsity level directly controllable [14]. These methods leave the trained network untouched and analyze it afterward. An alternative is to change how the network is trained so that its own units are easier to read.

A second body of work, developed for efficiency rather than interpretability, shows how to train networks that remain sparse. The lottery ticket hypothesis established that sparse subnetworks can train to the accuracy of the dense network [15], and a survey organizes the many ways of pruning and growing connections [16]. Dynamic sparse training methods keep the number of non-zero weights fixed while rewiring connections during training, by random regrowth [17], by momentum [18], or by gradient magnitude [19].

Guided Stochastic Exploration (GSE) belongs to this family but differs in how it computes sparsity [20]. Some earlier methods, such as RigL [19], periodically compute dense gradients to decide where to grow, and others maintain a dense copy of the weights. GSE is always sparse: it never materializes dense weights or dense gradients. To grow connections, it draws a random subset of the inactive ones and activates those in the subset with the largest gradient magnitude, which combines the unbiased exploration of random regrowth [17] with the guidance of gradient-based growth [19]. In its original evaluation on image classification, GSE matched or exceeded the accuracy of RigL and Top-KAST while its cost grew only linearly with the layer width.

This paper studies one question in the controlled setting of a toy model (Fig. 1): does training with sparse weights, sparse activations, or both make neurons correspond more closely to single features, and at what cost in capacity, meaning the number of the 128 features that the network can still reproduce accurately? We compare dense training with GSE, top-k activation sparsity, their combination, and a control whose sparse pattern is chosen at initialization and then held fixed. The comparison covers three feature probabilities and three widths, training sparse from the start against pruning a trained dense network, and a learning-rate sweep for the dense network and for GSE at 75% sparsity. The paper makes seven contributions. First, it defines two measures that separate capacity from cleanliness. Capacity is the number of features a network reproduces accurately, and cleanliness is measured by the number of features whose error rises when one neuron is removed. Second, it shows that sparse weights make neurons substantially cleaner at a capacity cost that depends on sparsity and width, small at 50% sparsity and absent at 128 neurons, and that this cleanliness is robust to the

learning rate whereas that of dense networks is not. Third, it shows that guided rewiring recovers capacity relative to a fixed sparse pattern at the same sparsity, but that the fixed pattern gives cleaner neurons, which is a negative result for the idea that rewiring itself adds interpretability. Fourth, it shows that the way connections are ranked matters in a small network, and it describes how the weights are organized during training, using visualizations [1]. Fifth, it shows that pruning an already trained dense network keeps its capacity but not its cleanliness, so the benefit for readability depends on training with sparse weights from the start. Sixth, it tests whether the picture changes when features come in related families, which bears on a recent challenge to the superposition account [10]. Dense neurons then mix several families, and sparse training makes neurons serve one family but rarely one feature. Seventh, it shows that the connectivity pattern GSE finds carries much of its capacity advantage but not its cleanliness advantage: when features are rare, networks retrained on that pattern recover much of the capacity and are less clean than GSE itself. Section II describes the task, the training arms and the metrics, Section III reports the experiments, Section IV discusses the results and their limits, and Section V concludes.

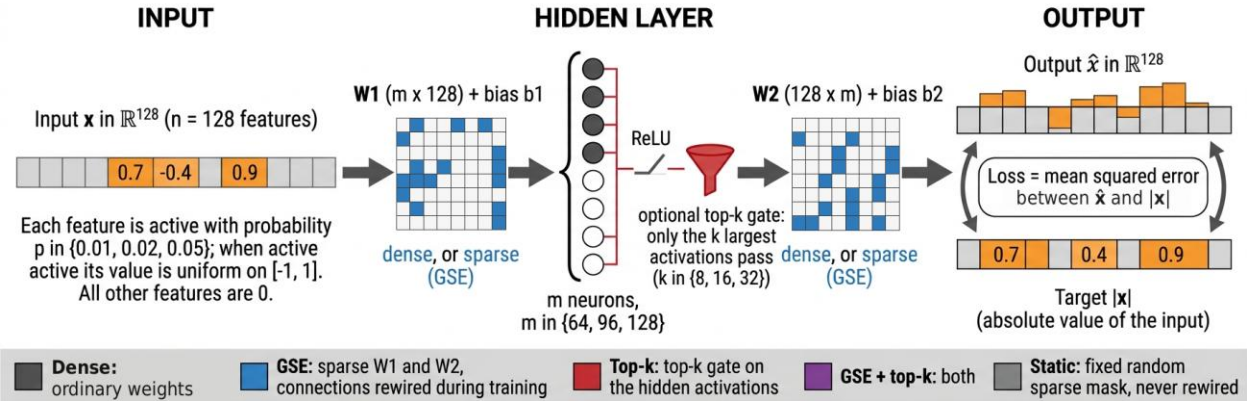


Fig. 1. Architecture of the toy model and the five training arms. A sparse input x with 128 features, each active with probability p and uniform on [-1, 1] when active, passes through a first weight matrix W1 and bias b1 to a hidden layer of m ReLU neurons (m = 64, 96 or 128, drawn here as eight circles), and then through a second weight matrix W2 and bias b2 to an output x-hat of 128 values. The network is trained to output the absolute value of the input, so the target is |x| and the loss is the mean squared error between x-hat and |x|. The small grids on the two arrows indicate that W1 and W2 are either dense or sparse, with blue squares standing for non-zero weights. In the top-k arms a gate placed after the ReLU (red funnel) lets only the k largest hidden activations pass (k = 8, 16 or 32), which is why only a few hidden neurons are drawn filled. The positions and heights in the input, output and target rows are schematic: the three active input features (0.7, -0.4 and 0.9) reappear in the target with absolute values 0.7, 0.4 and 0.9, and the output bars indicate an imperfect reconstruction. The color key at the bottom of the figure shows the arm colors used in all subsequent figures.

II. Method

A. Task and model

We follow the absolute-value task of Elhage et al. [4]. The input x has 128 features. Each feature is independently active with probability p, which we vary over 0.01, 0.02 and 0.05, and when active it takes a value drawn uniformly from [-1, 1]. Inactive features are zero. The target is the elementwise absolute value |x|. The network has one hidden layer of m ReLU neurons, with m = 64, 96 or 128, and

computes $\hat{y} = W2\ \mathrm{ReLU}(W1\ x + b1) + b2$, where W1 has m rows and n columns and W2 has n rows and m columns (Fig. 1). Absolute value is not linear, and a network that represents each feature without superposition needs two neurons for it, one for each sign. A network with m neurons can therefore represent about m/2 features cleanly, and representing more requires superposition. Unlike [4], we use a linear output layer and the unweighted mean squared error between $\hat{y}$ and $|x|$, so that all features are equally important. Weights are initialized with the Xavier uniform scheme [21] and biases with zeros.

B. Training Arms

We compare five arms that share the architecture and the training budget. The dense arm trains all weights. In the top-k arm, only the k largest hidden activations of each sample are passed to the second layer and the rest are set to zero, with k = 8, 16 or 32. The GSE arm keeps both W1 and W2 sparse from the first step to the last, at weight sparsity of 50, 75 and 87.5%. During training it prunes the active weights of smallest magnitude and grows the same number of new connections, chosen by the largest gradient among a random sample of inactive ones, so that the sparsity stays fixed while the connections change (Fig. 2). The biases are not masked. The combined arm applies GSE and top-k together. The static arm uses a random initial mask of the same kind as GSE and never changes it, which separates the effect of sparsity from the effect of rewiring.

Our GSE arm follows the unstructured method of Heddes et al. [20], reimplemented independently for this toy network. The initial connections are drawn at random, with the same density in W1 and W2, and the weights are initialized uniformly with a bound set by the actual number of inputs of each neuron. Every 100 steps a prune-grow round takes place. To grow, we draw a random subset of the inactive connections, as large as the number of active ones (in a matrix that is at most 50% sparse, this is the whole set of inactive connections), and activate the connections of the subset with the largest gradient magnitude on a fresh batch, which raises the number of active connections by a fraction α. New weights start at zero. To prune, we remove the smallest-magnitude active weights, excluding those just grown, which returns the count to its starting value. The fraction α follows a cosine curve from 0.2 to zero at 60% of training, after which the topology is frozen, and the weights continue to train on the fixed set of connections. The optimizer state of every changed connection is reset, and the mask is applied again after each update, so that unused weights stay exactly zero. In the main experiments we rank connections within each layer's weight matrix. Heddes et al. rank globally across layers, and we report that variant as a sensitivity check (Section III-C). Our implementation computes the dense gradient for the grow step and uses only the sampled subset of it, since the toy network is tiny. This selects the same connections as a sparse implementation would.

C. Training protocol

Every arm is trained for 20,000 steps with Adam optimizer at a batch size of 1024. The learning rate starts at 3×10^{-3} and decays to 1% of that value along a cosine schedule. Because the cleanliness and capacity of the dense network depend strongly on the learning rate, we repeated the comparison between the dense network and GSE at 75% sparsity at 10^{-3}, 3×10^{-3} and 10^{-2} (Section III-D). The main grid uses m = 64 neurons and five seeds per setting. The width experiments at m = 96 and m = 128 and the learning-rate sweep use three seeds. All reported numbers are means over seeds, and the error bars in the figures are standard deviations over seeds.

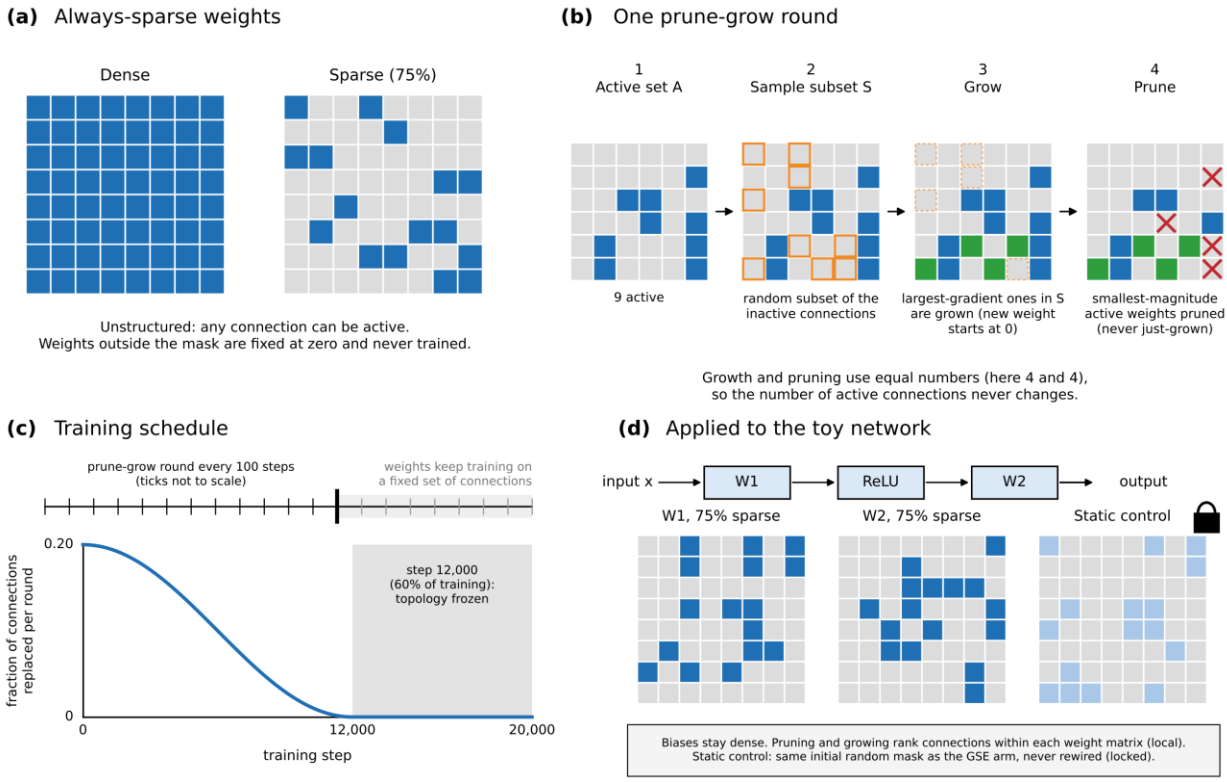


Fig. 2. Guided Stochastic Exploration (GSE) as used in this paper. (a) Always-sparse weights. The dense matrix has all 64 weights of an 8 × 8 grid active. The sparse matrix has 16 of 64 active (75% sparsity), at positions that can be anywhere, so that the pattern is unstructured. Weights outside the mask are held at zero and are never trained. (b) One prune-grow round, drawn on a 6 × 6 grid. The active set A has 9 connections. A random subset S of 9 inactive connections is drawn (orange outlines). The 4 connections of S with the largest gradient magnitude are grown (green) and start at zero, and the 4 previously active weights with the smallest magnitude are pruned (red crosses), so that the number of active connections stays at 9. (c) The schedule of one training run of 20,000 steps. A round takes place every 100 steps (the ticks are not drawn to scale), and the fraction of connections replaced per round falls along a cosine curve from 0.20 to zero at step 12,000 (60% of training), after which the topology is frozen, and the weights continue to train. (d) Use in the toy network. W1 and W2 are both 75% sparse, with 16 of 64 connections active in each grid, and connections are ranked within each matrix. The biases stay dense. Static control starts with a random mask of the same density and keeps it for the whole of training. The grids are illustrations and do not show a trained network.

D. Metrics

We measure capacity as the number of features that are represented. A feature is represented if the network's mean squared error on that feature, divided by the variance of the target $|x_i|$, is below 0.25 on a fixed held-out set of 50,000 samples. We also report the mean squared error over all features divided by the same variance, which we call the normalized loss.

We measure cleanliness by ablation. For each neuron, we remove its contribution to the output and record how much the squared error of each feature increases. The participation ratio of these increases, $(\Sigma v_i)^2 / \Sigma v_i^2$, is the effective number of features the neuron affects. It equals 1 when one feature is affected and n when all n features are affected equally. We call it the number of features affected per

neuron. We average it over the important neurons, those whose removal raises the total error by more than 0.1% of the target variance, weighing each neuron by that increase so that neurons that matter more count more. A neuron is clean if its ratio is below 1.5, and we report the fraction of important neurons that are clean. Ablation measures what a neuron does, not what it reads, so we also computed the participation ratio of each neuron's input weights and of the correlation between its activation and each feature. These give the same broad picture, so the tables report only the ablation measure.

E. Starting from a trained dense network

The main experiments train sparse networks from scratch. To ask whether sparsity imposed on an already trained network has the same effect, we first trained a dense network for 20,000 steps as above. From these weights we then took four arms. The dense arm trains for 10,000 further steps at a peak learning rate of 10^{-3} with the same cosine decay, without sparsity, and controls for the extra training. In the other arms the weights of smallest magnitude inside each weight matrix are removed to reach 50% or 75% sparsity. We report the pruned network as it is (one-shot), the pruned network fine-tuned for 10,000 steps with the mask fixed (static), and the pruned network fine-tuned for 10,000 steps with GSE rewiring, which starts from the same mask and uses the same rewiring schedule as before. For each seed all arms therefore start from the same trained weights. We ran this at 64 neurons for $p = 0.01$, 0.02 and 0.05 and at 128 neurons for $p = 0.01$, with five seeds each.

F. Related features

The toy above has independent features. To ask how related features change the picture, we added a variant of the data. The 128 features are split into 16 families of 8. For a co-occurrence level $c > 0$, a family is active with probability p/c and, when it is active, each of its members is active with probability c, so the members of a family tend to appear together, and every feature keeps its marginal probability p. The value $c = 0$ gives the independent features used so far. The network has 16 extra outputs, one per family, whose target is the sum of the absolute values of the family's members, so that a neuron that responds to several members of a family has a useful job, and the loss weights every output by the inverse of its variance. We trained all arms of Sections II-B and II-E at 64 neurons for $p = 0.01$ and 0.02 and $c = 0$, 0.25, 0.5 and 0.9, with three seeds each. We classify each important neuron by how its removal affects the features and the families. For each family, we add up the error increases of its eight members. The family participation ratio is the participation ratio of these 16 family totals, so it is the effective number of families the neuron affects. A neuron is clean if the number of features it affects is below 1.5, as before. It is grouped if it is not clean, but its family participation ratio is below 1.5, so it affects several features that all belong to one family. Otherwise, it is mixed, which means it affects features from more than one family.

G. Retraining using the mask GSE finds

GSE ends training with a network and a sparse connectivity pattern. To ask how much of its behavior the pattern carries by itself, we retrained networks on it. For each feature probability p, each sparsity (50% and 75%) and each seed, we first trained a GSE network from scratch as in Section II-B, with connections ranked within each matrix, and saved its final masks. We then trained four new networks for 20,000 steps with a mask that stays fixed. The rewind arm uses the final GSE mask and the initial weights of the GSE run; connections that GSE added during training start at zero, as they did in that run. The fresh arm uses the same mask with new random weights. The shuffled arm uses a random mask in which every neuron and every feature has the same number of connections as in the GSE mask, so it separates the specific wiring from the number of connections of each unit. The static arm uses the random mask from

which GSE started, which is the control of Section II-B. We also measured how often a neuron that reads a feature also writes back to it, that is, the fraction of connections from a feature to a neuron for which the connection from that neuron to the same feature is also present. We ran this at 64 neurons for p = 0.01, 0.02 and 0.05, with five seeds each.

III. Experiments and Results

A. Main grid

Table I reports the main grid at p = 0.01, where features are rarest and superposition is heaviest. The dense network represents 48.4 of the 128 features, well above the 32 that fit without superposition. Its neurons are strongly mixed: removing one neuron affects 13.6 features on average, and none of its important neurons is clean.

TABLE I. Main grid at p = 0.01 with 64 hidden neurons, averaged over five seeds. Features represented is out of 128. Features affected is the average number of features that a neuron affects, measured by ablation (Section II-D), and clean neurons are those that affect fewer than 1.5 features. Sparse arms are sparse in both weight matrices, with connections ranked within each layer's weight matrix.

Arm	Features represented	Normalized loss	Features affected per neuron	Clean neurons
Dense	48.4	0.641	13.6	0%
Top-k, k = 16	45.6	0.652	10.2	1%
Top-k, k = 8	38.2	0.652	3.1	18%
GSE, 50% sparse	45.2	0.636	5.5	5%
GSE, 75% sparse	26.2	0.671	2.3	50%
GSE, 50% sparse, with top-k, k = 16	41.2	0.636	3.5	16%
Static mask, 50% sparse	33.4	0.665	1.8	53%
Static mask, 75% sparse	13.6	0.732	1.1	92%

GSE at 50% sparsity represents 45.2 features, 7% fewer than the dense network, and removing one neuron affects 5.5 features, with 5% of neurons clean. At 75% sparsity, GSE represents 26.2 features, a neuron affects 2.3, and half of the neurons are clean.

The static mask is cleaner than GSE at the same sparsity but represents fewer features. At 50% it affects 1.8 features per neuron, with 53% of neurons clean, but represents only 33.4 features. At 75% it represents 13.6 features and is almost entirely clean (1.1 features per neuron, 92% clean), probably because it has too little capacity to use superposition at all.

Top-k activation sparsity changes little at k = 16, with 45.6 features represented and 10.2 affected per neuron. At k = 8 it affects 3.1 features, cleaner than GSE at 50% (5.5), but represents 38.2 features, 7 fewer than GSE. Combining GSE at 50% with top-k at k = 16 is cleaner than either alone (3.5 features per neuron) and represents 41.2 features, 4 fewer than GSE. The normalized losses of the dense network and GSE at 50% differ by less than 0.01 (0.640 and 0.636), so the cleaner neurons do not come from a worse overall fit.

Taken together, the main grid shows that sparsity trades capacity for cleanliness, and that GSE gives the best trade. At moderate sparsity it keeps nearly all of the dense network's capacity while making neurons much cleaner. At high sparsity it makes still more neurons clean, at the cost of about half the

capacity. The static mask and top-k activation sparsity give cleaner neurons than GSE at moderate sparsity, but they represent fewer features.

B. Capacity against cleanliness

Fig. 3 shows every arm of the main grid on the plane of capacity and cleanliness. The dense network and top-k with large k sit alone in the upper right, with the most features and the most mixed neurons. Every sparse method, whether the sparsity is on the weights, the activations or both, falls along one curve running to the lower left, so that more sparsity buys cleaner neurons but costs features. GSE at 50% lies at the upper end of that curve, close to the dense network in capacity and about two and a half times cleaner (5.5 against 13.6 features affected). Rewiring gives GSE more capacity than a static mask at the same sparsity, 45.2 against 33.4 features at 50% and 26.2 against 13.6 at 75%, but it does not make the neurons cleaner. The static mask at 50% has both more features and fewer features affected (33.4 and 1.8) than GSE at 75% (26.2 and 2.3), although it keeps twice as many weights. We therefore do not claim that rewiring makes neurons cleaner than a fixed sparse pattern does.

The benefit of sparsity also depends on how dense the features are (Table II). At p = 0.02 the dense network is already much cleaner than at p = 0.01, with 3.8 features affected per neuron, and GSE at 50% brings this to 2.4 at a cost of 5 features (47.8 to 42.6). At p = 0.05 the dense network affects only 2.1 features per neuron, GSE at 50% affects 1.9, and it gives up 8 of the 44 features to do so (43.6 to 36.0). Superposition is strongest when features are rare, and that is where sparse training helps most.

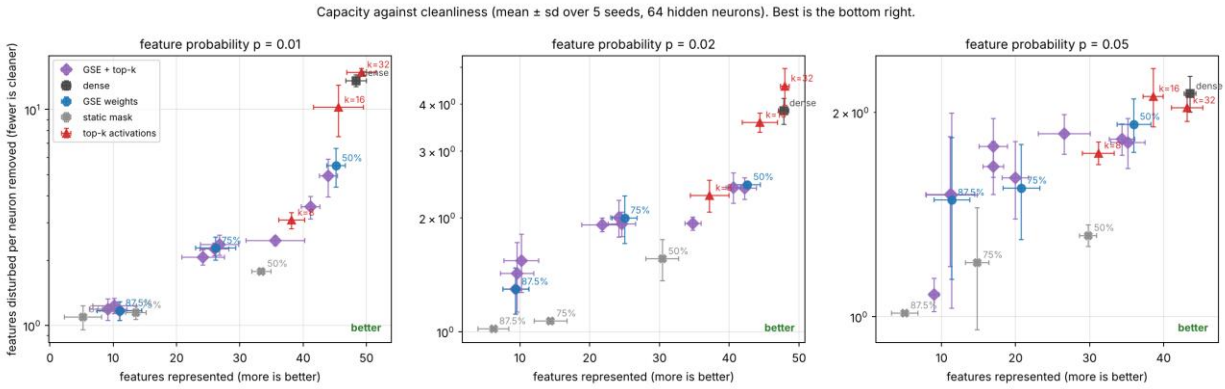


Fig. 3. Capacity against cleanliness for all arms at feature probabilities 0.01 (left), 0.02 (center) and 0.05 (right), with 64 hidden neurons. The horizontal axis is the number of features represented, so that further right means more capacity. The vertical axis, drawn on a logarithmic scale, is the average number of features whose error rises when one important neuron is removed, so that lower means cleaner and the best position in each panel is the bottom right. Each marker is a mean over five seeds and the bars are one standard deviation in each direction. Dark gray squares represent the dense network. Red triangles are top-k activation sparsity, labeled with k. Blue circles are GSE weight sparsity, labeled with the sparsity of the weights. Purple diamonds are GSE combined with top-k and are left unlabeled because they overlap the GSE points and lie close to them. Light gray crosses are the static mask, labeled with its sparsity. At p = 0.01 the dense network and top-k with k = 16 and 32 occupy the upper right (46 to 49 features, 10 to 15 per neuron), and the sparse arms form a curve toward the lower left, from GSE at 50% (45 features, 5.5 per neuron) through GSE at 75% (26, 2.3) and GSE at 87.5% (11, 1.2) to the static mask at 87.5% (5, 1.1). At p = 0.05 the points bunch together and many of the bars overlap, so the differences between arms there are small.

TABLE II. Effect of feature probability for the dense network and for GSE at two sparsities, with 64 hidden neurons and five seeds. Columns are as in Table I.

p	Arm	Features represented	Features affected per neuron	Clean neurons
0.01	Dense	48.4	13.6	0%
0.01	GSE, 50% sparse	45.2	5.5	5%
0.01	GSE, 75% sparse	26.2	2.3	50%
0.02	Dense	47.8	3.8	5%
0.02	GSE, 50% sparse	42.6	2.4	30%
0.02	GSE, 75% sparse	25.0	2.0	61%
0.05	Dense	43.6	2.1	28%
0.05	GSE, 50% sparse	36.0	1.9	46%
0.05	GSE, 75% sparse	20.8	1.5	79%

Taken together, every sparse method falls along one curve, on which more sparsity buys cleaner neurons at the cost of features. Rewiring moves GSE up the capacity axis without making its neurons cleaner. The benefit is largest when features are rare and nearly disappear as they become more common.

C. Scope of the ranking

The original GSE method ranks connections across all layers of the network together, so that a weight in one layer competes with the weights of every other layer for the same budget. We tested this global ranking at p = 0.05 with five seeds (Table III). In the toy network it moves most of the budget into one weight matrix. At 75% sparsity the first matrix W1 ended with 1.2 to 1.5% of its connections active and W2 with 48.5 to 48.8%, against 25% each in the static control and in the per-matrix version, so that W1 had about 110 active weights for 128 features. This is why global GSE at 75% sparsity represents 15.8 features, hardly above the static mask (14.8) and well below per-matrix GSE (20.8), and why at 87.5% it has perfectly clean neurons (1.0 features affected) but represents only 8.4 features. At 50% sparsity the two scopes behave alike (35.0 and 36.0 features). A network with only two weight matrices gives a global ranking little room to balance itself, and a deeper network, in which many layers compete, may behave differently. We therefore use per-matrix ranking for all other results, and we regard the global result as a limit on how far our conclusions about the capacity advantage of GSE extend to the original method.

TABLE III. Scope of the ranking at p = 0.05 with 64 hidden neurons and five seeds. Each cell gives the number of features represented followed by the average number of features affected per neuron. Per matrix ranks connections inside each weight matrix, as in the main experiments, and global ranks them across both matrices together, as in the original method.

Sparsity	Static mask	GSE, per matrix	GSE, global
50%	29.8 / 1.3	36.0 / 1.9	35.0 / 2.0
75%	14.8 / 1.2	20.8 / 1.5	15.8 / 1.5
87.5%	5.0 / 1.0	11.4 / 1.5	8.4 / 1.0

Taken together, ranking connections across both matrices removes much of GSE's advantage in this network. It moves nearly all the budget into the output matrix and leaves the input matrix almost empty, so the network represents fewer features than with per-matrix ranking. We therefore rank within each matrix, and our conclusions apply to that variant.

D. Width and learning rate

The main grid fixes the width at 64 hidden neurons and the learning rate at 3×10^{-3}. We therefore repeated the comparison with wider hidden layers of 96 and 128 neurons (Table V) and swept the learning rate at 128 neurons (Table IV). The cleanliness of GSE is robust to the learning rate, while the dense network is not. Across a tenfold range of learning rates, GSE at 75% sparsity keeps between 1.9 and 2.4 features affected per neuron, whereas the dense network goes from 20.3 at the lowest learning rate to 7.2 at the highest. A better-tuned dense network is therefore cleaner, but not as clean: at its best learning rate (10^{-2}) it represents 101 features, and each neuron still affects 7.2 of them, almost four times as many as in the GSE network. The capacity comparison also depends on the learning rate. At 10^{-3} the dense network is under-trained, and GSE at 75% represents 85.0 features against 65.7. At 3×10^{-3} it represents 81.7 against 92.7, which is 12% fewer, and at 10^{-2} it represents 63.7 against 101.0, which is 37% fewer. The capacity of GSE itself fell as the learning rate rose, which we did not investigate. A fair comparison between a sparse and a dense network therefore needs a separate learning rate for each.

TABLE IV. Learning-rate sweep for the dense network and for GSE at 75% sparsity, with 128 hidden neurons, p = 0.01 and three seeds. Each cell gives the number of features represented followed by the average number of features affected per neuron.

Learning rate	Dense	GSE, 75% sparse
0.001	65.7 / 20.3	85.0 / 2.4
0.003	92.7 / 16.0	81.7 / 2.2
0.01	101.0 / 7.2	63.7 / 1.9

TABLE V. Effect of width at a learning rate of 3×10^{-3} with three seeds. Cells are as in Table IV.

Neurons	p	Dense	GSE, 50% sparse	GSE, 75% sparse
96	0.01	72.3 / 16.1	69.3 / 3.9	55.0 / 2.4
96	0.02	71.7 / 3.1	70.0 / 2.5	50.3 / 1.8
128	0.01	92.7 / 16.0	96.3 / 3.5	81.7 / 2.2
128	0.02	95.7 / 3.2	94.7 / 2.2	77.3 / 2.0

Width changes the picture in favor of sparse training (Table V). At p = 0.01, GSE at 50% sparsity represents 45.2 features at 64 neurons against 48.4 for the dense network, 69.3 against 72.3 at 96 neurons, and 96.3 against 92.7 at 128 neurons. At 128 neurons it therefore matches the capacity of the dense network, and its neurons are 4.5 times cleaner (3.5 features affected against 16.0). GSE at 75% sparsity keeps 54%, 76% and 88% of the capacity of the dense network at 64, 96 and 128 neurons. At p = 0.02 with 128 neurons, GSE at 50% represents 94.7 features against 95.7 for the dense network, with cleaner neurons (2.2 features affected against 3.2). The static mask at 50% and 128 neurons represents 81.3 features at p = 0.01 and 77.3 at p = 0.02, with 1.7 features affected in both cases, so guided rewiring again buys capacity (96.3 and 94.7 features) at the price of less clean neurons (3.5 and 2.2). These width results use the shared learning rate of 3×10^{-3}, which is not the best one for the dense network (Table IV), and we did not test whether GSE at 50% would also gain from a larger step.

Taken together, the cleanliness of GSE is stable across learning rates, whereas that of the dense network is not, and the capacity cost of sparsity shrinks as the network widens. With enough hidden neurons, GSE at moderate sparsity matches the capacity of the dense network with much cleaner neurons, although the dense network is not at its best learning rate in this comparison.

E. Starting from a trained dense network

Table VI starts from a trained dense network instead of from scratch. Pruning a trained network did not make its neurons clean. At p = 0.01 with 64 neurons, the dense network after the extra training represents 48.6 features, and 13.2 are affected per neuron. At 75% sparsity the pruned and fine-tuned networks represent 47.2 (static mask) and 48.0 (GSE) features, nearly as many as the dense network, but their neurons still affect 12.0 and 11.2 features. Trained from scratch at the same sparsity, GSE represents 26.2 features and affects 2.3 per neuron. Starting from a trained network therefore keeps about 1.8 times the capacity of training sparse from the start, and none of its cleanliness. Pruning without fine-tuning loses capacity at high sparsity (16.8 features at 75% with 64 neurons, and none with 128), and fine-tuning recovers it. With 128 neurons the picture is the same. GSE at 75% fine-tuned from the dense network represents 83.4 features and affects 5.9 per neuron, against 81.7 and 2.2 when trained from scratch, so the capacity is about the same and the neurons are about 2.7 times less clean. At 50% sparsity GSE fine-tuning gives 87.8 features and 12.5 affected per neuron, against 93.2 and 14.2 for the dense network. Of the two ways of fine-tuning, GSE keeps more capacity than the static mask with 128 neurons (83.4 against 64.6 features), as it does from scratch, and the static mask is cleaner (4.4 against 5.9). With 64 neurons GSE is slightly better on both measures. At p = 0.02 and 0.05 every arm matches the dense network (47.8 features and 3.7 affected per neuron at p = 0.02), because the dense network represents only 44 to 48 of the 128 features there. We did not check this, but it is consistent with the pruned weights belonging to features that the network does not represent. GSE does rewire when it starts from a trained network: at p = 0.01 and 75% sparsity it keeps 84% of its initial connections with 64 neurons and 61% with 128. Continuing to train the dense network for the same number of extra steps changes little (13.6 to 13.2 features affected at 64 neurons), so the extra training does not explain these results.

TABLE VI. Pruning a trained dense network, with five seeds. Each cell gives the number of features represented followed by the average number of features affected per neuron, as in Table I. Within each seed all networks start from the same trained dense network and receive 10,000 further steps. Dense continues without sparsity. The three columns at 75% sparsity prune by magnitude and are then used as they are (one-shot), fine-tuned with a fixed mask (static) or fine-tuned with GSE. The last column repeats GSE trained sparse from the start.

Neurons	p	Dense	One-shot 75%	Static 75%	GSE 75%	GSE 75% from scratch
64	0.01	48.6 / 13.2	16.8 / 12.5	47.2 / 12.0	48.0 / 11.2	26.2 / 2.3
64	0.02	47.8 / 3.7	47.8 / 3.8	47.8 / 3.7	47.8 / 3.7	25.0 / 2.0
64	0.05	43.6 / 2.2	43.6 / 2.1	43.6 / 2.2	43.6 / 2.2	20.8 / 1.5
128	0.01	93.2 / 14.2	0.0 / 11.9	64.6 / 4.4	83.4 / 5.9	81.7 / 2.2

Taken together, pruning and fine-tuning a trained dense network keeps its capacity but not its cleanliness. Networks that are pruned after dense training stay about as entangled as the dense network, whereas GSE, which trains sparse from the start, produces much cleaner neurons at a lower capacity. Clean neurons therefore depend on training with sparse weights from the beginning, and the route a network takes decides where it ends up between capacity and cleanliness.

F. Inside the network

The numbers above say that sparse networks have cleaner neurons. Figs. 4 to 7 show what this looks like inside one network at p = 0.01 with 64 neurons, for a single seed (seed 4) whose results are close to the five-seed means: it represents 49 features with the dense network and 26 with GSE at 75%. The heat

maps of the first-layer weights (Fig. 4) show that in the dense network a few features are read by many neurons at once, which appear as vertical stripes, and that only a handful of neurons have a clear main feature. In both GSE arms a diagonal is already visible at step 200 and is clear by step 1000, which means that each neuron comes to read one dominant feature. Little weight remains away from the diagonal, although a few scattered off-diagonal weights stay, more than in the static arm. Top-k alone resembles the dense network, with stripes and with a set of features that the network has largely dropped.

The connection patterns (Fig. 5) show how the GSE networks reach this arrangement. By the end of training, the connections added to the first layer number 71% of the initial connections in the GSE network and 73% in the combined network, so most of the initial mask has been replaced, and substantial rewiring is already visible at step 200. The diagonal in Fig. 4 therefore forms while connections are being moved. The added connections are not spread evenly: by the end of training, horizontal orange bands show neurons that have gained many first-layer connections, while others have lost most of theirs. The static control keeps its initial mask and nevertheless shows a diagonal, so a good arrangement can also be obtained from a random fixed mask, at the price of a much lower capacity at the same sparsity.

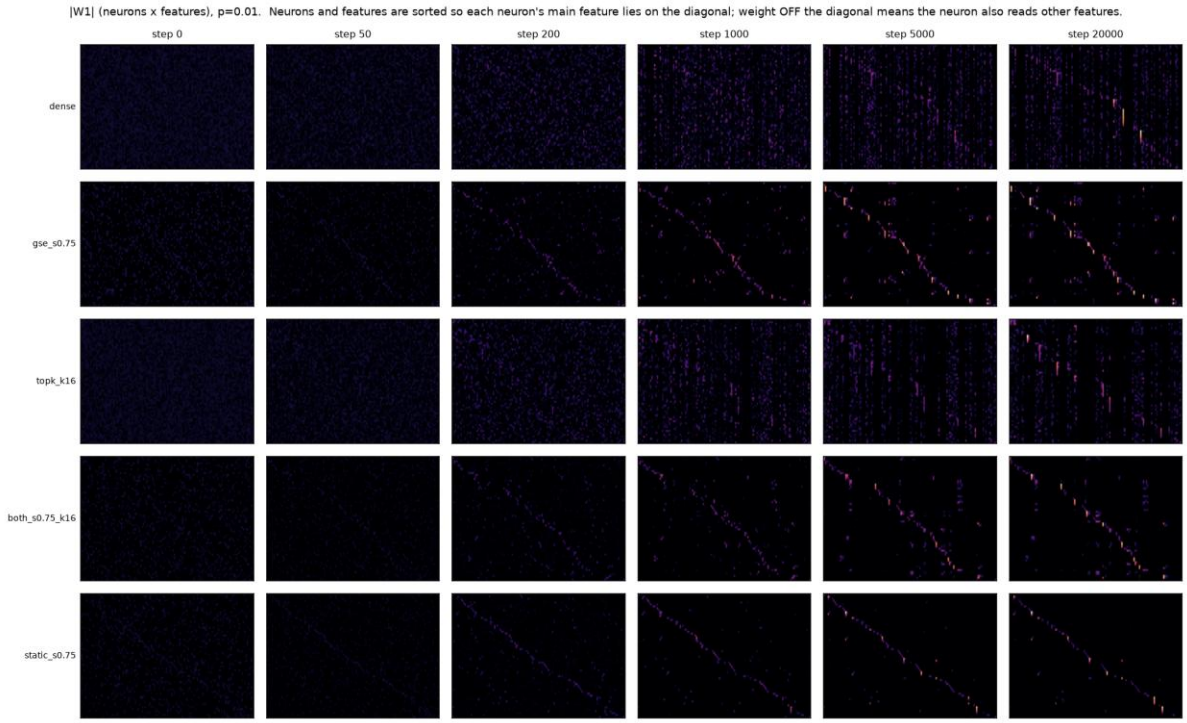


Fig. 4. Evolution of the first-layer weight magnitudes |W1| at p = 0.01 for seed 4 with 64 hidden neurons. The rows are the dense, GSE 75%, top-k (k = 16), GSE 75% with top-k (k = 16) and static 75% arms. The columns are training steps 0, 50, 200, 1000, 5000 and 20000. In each panel the 64 neurons (vertical axis) and the 128 features (horizontal axis) are ordered once, using the final weights, so that each neuron's strongest feature lies on the diagonal, and the same ordering is used at every step. Brighter pixels are larger weights, and the color scale is shared across the columns of a row and set by the largest final weight in that row. A neuron that reads a single feature shows one bright pixel on the diagonal, and weight away from the diagonal means that the neuron also reads other features. Bright vertical stripes are features read by many neurons at once, and dark columns are features with almost

no weight. The static arm shows a thin diagonal from step 0, because its mask is fixed and the ordering computed from the final weights already reveals the pattern in the mask.

The single-neuron views (Figs. 6 and 7) make the difference concrete. Of the six most important neurons of the dense network, three (neurons 58, 50 and 47) read six to eight features at comparable weights, with no dominant feature, and the other three (neurons 0, 42 and 25) have one main feature (feature 82 for neurons 0 and 42, feature 71 for neuron 25) together with at least five others at roughly 10% to 30% of its weight. Their outputs are spread over many features as well, and removing one of them affects between 8.4 and 19.1 features. In the GSE network four of the six most important neurons read one or two features strongly: feature 93 (neuron 58), feature 25 (neuron 57), features 4 and 122 (neuron 19) and features 96 and 81 (neuron 34). Their outputs concentrate on the same few features, the inputs that excite them most are dominated by those features, and removing one affects between 1.1 and 2.8 features. In the GSE network the picture is not strictly one feature per neuron. Neuron 42 reads three features (71, 81 and 96), neuron 25 reads four at similar weights although it writes mostly to feature 19, neurons 42 and 34 share features 81 and 96, and neurons 19 and 25 share features 4 and 122. GSE therefore yields neurons that are mostly dedicated to one or two features rather than a one-to-one map.

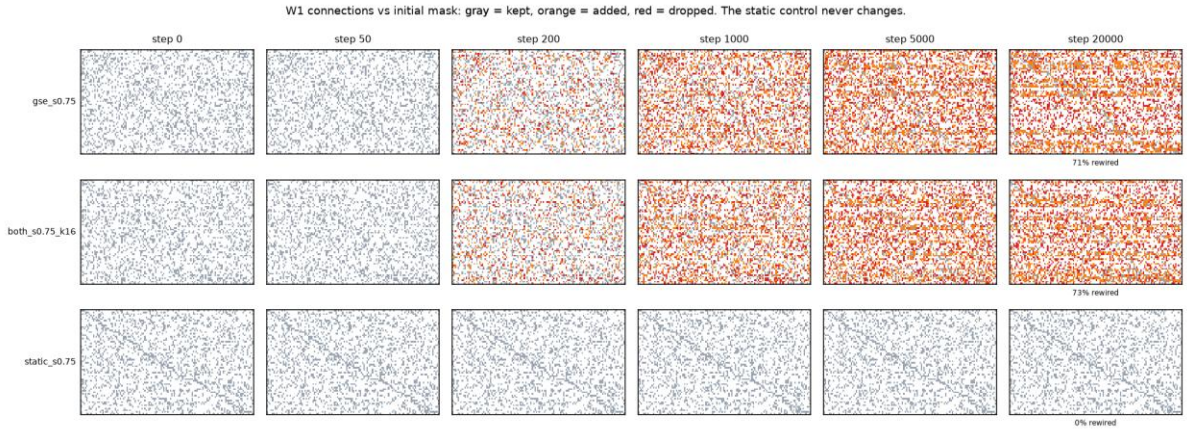


Fig. 5. Rewiring of the first-layer connections in the sparse arms at p = 0.01 relative to the initial mask, for the same seed, ordering and steps as Fig. 4. Gray marks a connection that was present at initialization and is still present, orange marks a connection that has been added since initialization, red marks a connection that was present at initialization and has since been dropped, and white marks no connection. The rows are GSE 75%, GSE 75% with top-k (k = 16) and static control. The label under the final column gives the number of added connections as a fraction of the initial number of connections: 71% for GSE, 73% for GSE with top-k and 0% for the static control, whose pattern never changes. Substantial rewiring is visible by step 200, and the topology is frozen from step 12,000.

Figs. 6 and 7 show one neuron per row. The first column gives the eight largest input weights of the neuron in W1, and the second the eight largest output weights in W2. The third column shows which features are present in the 0.5% of held-out samples that excite the neuron most, as the factor by which each feature is over-represented compared with chance (75 means the feature is active 75 times more often than its base rate p). The title of each row gives the neuron index and its ablation participation ratio, the effective number of features whose error rises when the neuron is removed.

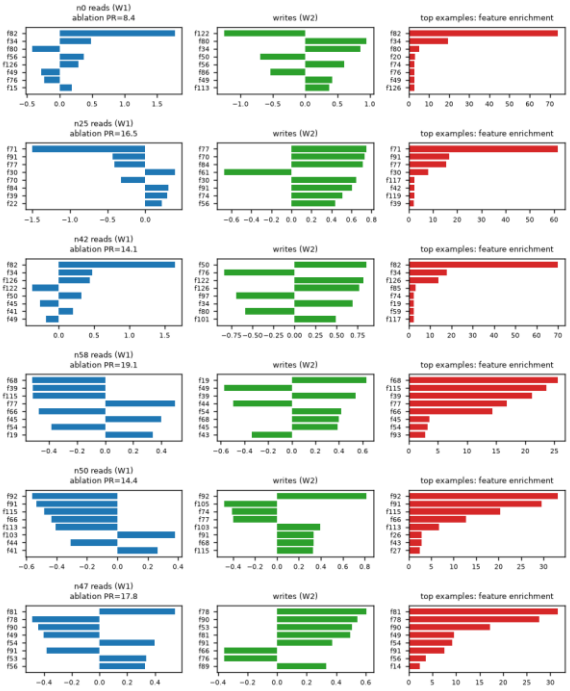


Fig. 6. The six most important neurons of the dense network at p = 0.01 with 64 hidden neurons (seed 4). The ablation ratios range from 8.4 to 19.1.

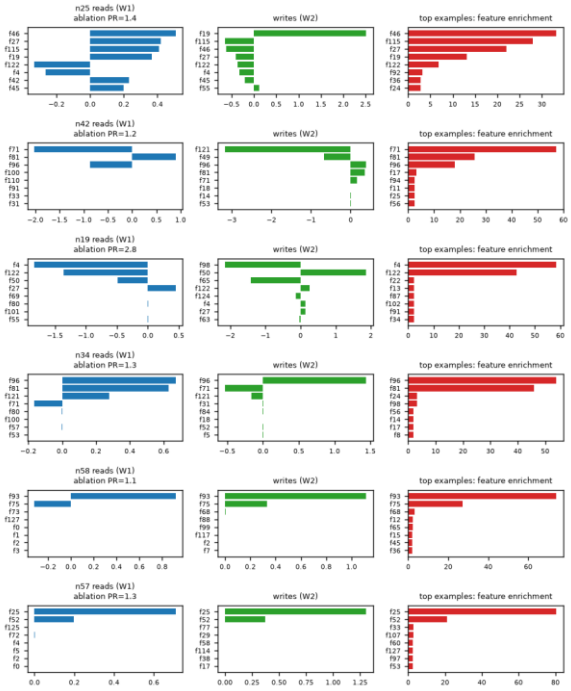


Fig. 7. The same view for the GSE network at 75% sparsity. The ablation ratios range from 1.1 to 2.8.

Taken together, the figures show the same result at the level of single weights. In the dense network, many neurons read many features through shared patterns, and some have no dominant feature. GSE neurons instead come to read one or two features each, and this structure forms while connections are still being replaced. The arrangement is dedicated rather than strictly one-to-one, because some GSE neurons share features.

G. Related features

Table VII and Fig. 8 report this test with the arms used before. The dense network represented all 16 families at every setting. At p = 0.01 all its important neurons were mixed once the features were related (c ≥ 0.25) and 91% were mixed with independent features, and a neuron affected between 6.6 and 7.5 families when c was at least 0.25, against 4.3 when c = 0. Its largest family accounted for only 25% to 29% of the damage caused by removing the neuron. At p = 0.02 the numbers were lower, 2.8 to 4.8 families against 2.0, and at c = 0.25 about one fifth of its neurons were grouped. Related features therefore did not make the dense neurons line up with families, and if anything, they mixed more families than independent features did.

Sparse training changed this at every co-occurrence level. GSE at 75% sparsity brought the family participation ratio down to between 1.3 and 1.6, left 9% to 33% of the neurons mixed and made 56% to 88% of them grouped, so most neurons served one family. Few were clean (0% to 17%). The fixed random mask gave the cleanest neurons, 24% to 85% of them clean, but at 75% sparsity it represented at most two of the 16 families, so these neurons belong to a network that has largely not learned the

task. Top-k with k = 8 left 26% to 59% of the neurons mixed while representing all families, and top-k with k = 16 left 90% to 100% mixed when c was at least 0.5.

TABLE VII. Related features, with 64 neurons, family outputs and three seeds. Each cell gives the family participation ratio, the effective number of families whose error rises when one neuron is removed, followed in parentheses by the fraction of important neurons that are mixed. The co-occurrence level c = 0 gives independent features. Dense→GSE 75% prunes the trained dense network by magnitude and fine-tunes it with GSE. Static 75% represents at most two of the 16 families.

p	c	Dense	GSE 75%	Static 75%	Top-k (k = 8)	Dense→GSE 75%
0.01	0	4.3 (0.91)	1.6 (0.33)	1.1 (0.03)	1.7 (0.42)	2.9 (0.79)
0.01	0.25	6.6 (1.00)	1.6 (0.26)	1.2 (0.05)	1.7 (0.43)	2.4 (0.77)
0.01	0.5	7.5 (1.00)	1.6 (0.28)	1.2 (0.08)	2.1 (0.59)	1.7 (0.53)
0.01	0.9	6.8 (1.00)	1.6 (0.29)	1.3 (0.16)	1.8 (0.55)	1.6 (0.45)
0.02	0	2.0 (0.58)	1.4 (0.18)	1.1 (0.01)	1.5 (0.26)	2.0 (0.51)
0.02	0.25	2.8 (0.79)	1.5 (0.12)	1.1 (0.03)	1.5 (0.27)	1.9 (0.57)
0.02	0.5	4.8 (1.00)	1.3 (0.09)	1.2 (0.04)	1.3 (0.29)	1.7 (0.52)
0.02	0.9	4.4 (1.00)	1.5 (0.17)	1.4 (0.09)	1.8 (0.49)	1.7 (0.55)

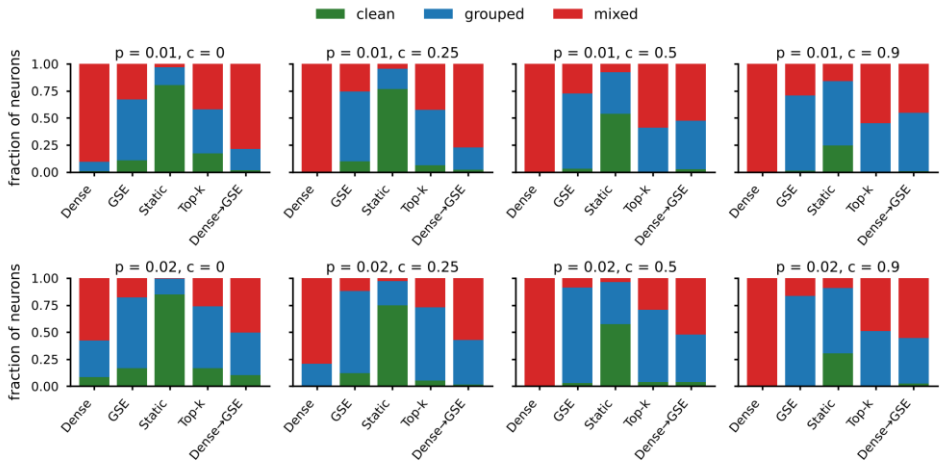


Fig. 8. Neuron types among the neurons whose removal matters, with 64 neurons and family outputs. A neuron is clean if its removal affects about one feature, grouped if it affects several features that all belong to one family, and mixed if it affects features from more than one family. Rows give p = 0.01 and 0.02, columns give the co-occurrence level c = 0, 0.25, 0.5 and 0.9, and bars are means over three seeds. GSE, Static and Dense→GSE use 75% sparsity and Top-k uses k = 8. Static represents at most two of the 16 families.

Pruning the trained dense network did not give the same result. GSE at 75% sparsity started from the dense weights represented nearly all families but left 45% to 79% of the neurons mixed, against 9% to 33% when GSE was trained sparse from the start. The gap was smallest at c = 0.9 and p = 0.01, where the two arms left 45% and 29% mixed.

Two checks bear on how far these results can be trusted. At c = 0.5 and a peak learning rate of 10^{-3} the dense result was unchanged (6.5 to 7.8 families affected, all neurons mixed), but GSE was less clean than at 3×10^{-3}. At p = 0.01 and 50% sparsity 15% of its neurons were grouped against 50%, and at 75%

sparsity 57% against 69%, so the effect of sparse training depends on the learning rate more than the dense result does. The experiment also uses one width, one family size and three seeds.

Taken together, related features do not make dense neurons line up with families, but sparse training does group neurons by family without making them single feature. Dense neurons mix several families, whereas GSE leaves most neurons reading a single family. Pruning a trained dense network leaves many more neurons mixed, so the route matters here as well. Because the effect weakened at a lower learning rate, we regard it as preliminary.

H. Retraining on the mask GSE finds

Table VIII compares the networks trained on the final GSE mask with the GSE run that found it. We start with p = 0.01, where superposition is strongest. At 75% sparsity the mask carries most of the capacity. Trained from the original weights, the network on the GSE mask represents 26.4 features, as many as GSE itself (26.8), and from new random weights it represents 21.6. The random masks represent only 13.6, whether they are the static mask or the shuffled one. The specific wiring is therefore worth about 8 features, and the original weights about 5 more. The wiring has a recognizable form: 46% of the first-layer connections of the GSE mask have a matching connection from the same neuron back to the same feature in the second layer, against 25% for a random mask, so a neuron tends to write to the features that it reads.

TABLE VIII. Retraining on the mask found by GSE, with 64 hidden neurons, five seeds and connections ranked within each matrix. Each cell gives the number of features represented followed by the average number of features affected per neuron, as in Table III. GSE is the run that found the mask; it is a separate run from Table I, so its values differ slightly. Rewind and Fresh train a fixed copy of its final mask from its initial weights and from new random weights, Shuffled uses a random mask with the same number of connections at every neuron and every feature, and Static uses the random mask from which GSE started.

p	Sparsity	GSE	Rewind	Fresh	Shuffled	Static
0.01	50%	43.4 / 5.4	45.6 / 6.9	46.0 / 7.1	45.4 / 7.9	34.2 / 1.8
0.01	75%	26.8 / 2.3	26.4 / 2.9	21.6 / 2.7	13.6 / 1.3	13.6 / 1.1
0.02	50%	42.2 / 2.5	42.2 / 2.5	35.6 / 1.8	30.2 / 1.7	30.6 / 1.6
0.02	75%	25.4 / 1.9	24.8 / 2.4	18.6 / 1.6	12.0 / 1.2	14.6 / 1.1
0.05	50%	35.8 / 2.0	32.0 / 1.7	27.2 / 1.4	26.0 / 1.2	30.4 / 1.3
0.05	75%	22.2 / 2.0	20.4 / 1.6	16.2 / 1.3	8.0 / 1.2	14.6 / 1.2

At 50% sparsity the mask carries capacity in a different way. All three networks trained on a mask derived from GSE represent 45 to 46 features, against 34.2 for the static mask and 43.4 for GSE itself, and the shuffled mask does as well as the exact one. A random mask with the same number of connections at each neuron and feature therefore reproduces the capacity of GSE here, which suggests that at this sparsity the gain over the static mask comes from how many connections each neuron and feature has, not from which ones.

The cleanliness of GSE did not transfer at p = 0.01. At 50% sparsity, networks retrained on its mask, or on a random mask with the same number of connections per neuron, affect 6.9 to 7.9 features per neuron, against 5.4 for GSE. At 75% sparsity, the two networks retrained on its exact mask affect 2.9 and 2.7 features per neuron, against 2.3. They remain much cleaner than the dense network (13.6 features affected in Table I). The static and shuffled masks at 75% sparsity are cleaner still (1.1 and 1.3), but they represent only 13.6 features.

At p = 0.02, the networks that start from the original weights reproduce GSE at 50% sparsity (42.2 features and 2.5 affected per neuron for both), whereas new weights lose about 7 features at both sparsities (35.6 against 42.2, and 18.6 against 25.4) and make the neurons cleaner (1.8 and 1.6 features affected). At p = 0.05 the mask transfers poorly. At 50% sparsity the networks with new weights or a shuffled mask represent 27.2 and 26.0 features, fewer than the static mask (30.4), and even the rewound network recovers only 32.0 of the 35.8 features of GSE. Whenever a retrained network is clearly cleaner than GSE, it also represents fewer features, so these networks move along with the trade-off between capacity and cleanliness of Fig. 3 and do not beat it. These are means over five seeds at 64 neurons, and differences of a feature or two are within the variation between seeds in the main grid.

Taken together, the connection pattern that GSE finds carries much of its capacity but not its cleanliness. A network trained on that pattern recovers most of GSE's capacity when it starts from the original weights, and part of it when it starts from new weights. A random pattern with the same number of connections per neuron matches GSE only when sparsity is moderate and features are rare. The retrained networks are far cleaner than the dense network but less clean than GSE when features are rare. The pattern and the path that finds it both contribute to capacity, and weight sparsity itself accounts for most of the cleanliness.

IV. Discussion

A. Why sparse weights make neurons cleaner

The simplest explanation of our main result is a counting argument: if every neuron may connect to only a few features, then every neuron can affect only a few features. The numbers show that this cannot be the whole story. At 75% sparsity a neuron of the 64-neuron network reads on average 32 of the 128 features, a quarter of the entries of its row of W1 and writes to on average 32 of the 128 outputs, a quarter of the entries of its column of W2. The budget thus allows a neuron to read and write about 32 features, whereas the observed neurons affect 2.3 features on average at p = 0.01. At 50% sparsity the budget allows 64 features per neuron, and the observed value is 5.5, and at 87.5% it allows 16 and the observed value is 1.2. The constraint is therefore loose by an order of magnitude, and the networks use only a small part of the freedom it leaves. We interpret this as an effect on the solution that optimization finds, not a direct cap on what each neuron can do: with a fixed budget of connections, a connection spent on a secondary feature is taken away from a primary one, and the gradient favors concentrating the budget where it reduces the error most. Starting from a trained dense network (Section III-E) gives a partial test of this reading, discussed below, but we have not isolated the effect fully, and masking only W1 or only W2 would show which side matters.

Static control supports part of this reading and qualifies another part. A random fixed mask is cleaner than GSE at the same sparsity (1.8 against 5.5 features affected at 50%, and 1.1 against 2.3 at 75%), which shows that a fixed sparse structure is enough to make neurons clean and that rewiring is not needed for cleanliness. What rewiring adds is capacity: at 50% sparsity GSE represents 45.2 features against 33.4 for the static mask, because choosing which connections to keep lets the network place its limited weights where the features are. The extra capacity is not free. A plausible reading is that a fixed random mask cannot give every feature a neuron that suits it, so it leaves some features unrepresented, whereas a rewired network represents more features and lets some of them share neurons again. Guided rewiring therefore gains capacity at a given sparsity, and the gain in cleanliness comes from sparsity itself.

The experiment that starts from a trained dense network supports the reading that cleanliness is a property of the solution that training finds, and not of the constraint alone. If sparsity only limited what each neuron can read, pruning a trained network to the same sparsity should give clean neurons too. It does not: the pruned and fine-tuned networks keep the mixed neurons of the dense network (11 to 12 features affected with 64 neurons, against 2.3 from scratch). The same experiment shows that training dense first does not give clean neurons and high capacity together, because the trained network keeps its capacity only by keeping its superposition. Cleaner neurons and higher capacity therefore pull in different directions, and the route taken decides where a network ends up between them.

Retraining on the mask that GSE finds (Section III-H) refines this picture. At 75% sparsity and $p = 0.01$ the mask alone gives back much of the capacity (21.6 features from new weights, against 13.6 for a random mask) and the original weights give back the rest (26.4), so what GSE finds is a pattern of connections that suits the features, and the path that finds it also matters. The pattern is not what makes the neurons clean: the networks trained on it are less clean than GSE itself at $p = 0.01$ (6.9 to 7.9 features affected per neuron at 50% sparsity against 5.4), although far cleaner than the dense network (13.6). Cleanliness therefore comes mainly from the sparsity of the connections, while the pattern and the path add capacity.

B. Capacity, cleanliness and the meaning of a clean neuron

The trade-off curve in Fig. 3 raises a question that our experiments only partly answer. A network that represents fewer features has fewer features to spread over its neurons, so some of the cleanliness of the sparse networks may simply reflect their lower capacity. Some comparisons are free of this concern. At 128 neurons and $p = 0.01$, GSE at 50% sparsity represents at least as many features as the dense network (96.3 against 92.7) with neurons that are 4.5 times cleaner (3.5 against 16.0 features affected), so there the cleanliness is not a by-product of lower capacity, although the dense network is not at its best learning rate in this comparison. Where the sparse network has clearly lower capacity, for example at 75% sparsity, we cannot separate the two effects, because the dense network appears at a single capacity for each width in our grid. A dense baseline of matched capacity, obtained by reducing the width until the number of represented features equals that of the sparse network, would settle whether sparsity does anything beyond lowering capacity in those settings. Without it, our claim there is that sparse training reaches a point on the capacity and cleanliness plane that our dense networks did not reach, not that no dense network can be as clean.

The notion of a clean neuron also deserves care. In the absolute-value task a feature represented without superposition needs two neurons, one for each sign, so the ideal outcome is not one neuron per feature but two neurons that each serve a single feature. A participation ratio near one therefore describes a neuron dedicated to one feature, which is weaker than a one-to-one map between neurons and features, and the GSE neurons in Fig. 7 show this: some share a feature pair or read the same feature as another neuron. Our measure is also computed from the effect of removing a neuron on the output, so it captures what a neuron does and may differ from what it reads. The two views agree broadly in our tables, but in principle a neuron could read many features and write to few, or the reverse. Finally, the threshold of 1.5 on the participation ratio is a convention, and the continuous average in the tables does not depend on it.

C. Relation to superposition and to sparse autoencoders

In the account of Elhage et al. [4], superposition is a compression strategy in which a network trades interference between features for capacity and does so most readily when features are sparse. Sparse

connectivity changes the terms of this trade. When each neuron reads few inputs, fewer features share a neuron, and interference becomes local rather than spread across all the neurons. The dense network at p = 0.01 uses the freedom to mix freely and ends up with neurons that affect about 14 features each. Our results suggest that a connectivity constraint makes the compression more structured, so that the sparse network reproduces fewer features but with neurons that are easier to read.

Sparse autoencoders [11]–[14] take the opposite route. They leave the network as trained and search for a sparse dictionary in its activations. The two approaches are complementary rather than competing. A sparsely trained network might need a smaller dictionary, or none, and a sparse autoencoder trained on its activations might find cleaner features than one trained on a dense network. The weight-sparse transformers of Gao et al. [22] point in the same direction at a much larger scale. Our contribution relative to that work is a setting in which the features are known, so that capacity and cleanliness are measured rather than inferred, and in which sparsity comes from training methods designed for efficiency, so that any interpretability benefit could come together with an efficiency benefit.

A recent essay by Cammarata [10] challenges the view that superposition explains most polysemanticity in real networks. It separates two senses of the word. One represents more features than dimensions, as in Elhage et al. [4], which the essay sets aside without calling it wrong. The other is arbitrary packing, in which unrelated things share a neuron because they rarely interfere, and this is the sense it argues against. Its main claims are that the canonical examples of arbitrary packing were misread, because weak interpretability tools and architectures that learned poorly made neurons look mixed, and that most of the polysemanticity that remains is recursive generalization, in which a neuron's meaning grows outward step by step into related territory. It therefore treats arbitrary packing as a hypothesis of last resort. These claims rest on preliminary audits of language models and on retrained vision models. The essay is informal and has not been peer reviewed, and we have not tested its claims on real networks.

The essay also discusses the toy models of Elhage et al. [4]. It finds no problems with their experiments and accepts that packing occurs in the toy world, where features are independent and each is an end. Its question is which world real networks are in, meaning how much structure real data has relative to the room the network has. Our main experiments sit in the toy world. The features are independent, so nothing is shared between them for a neuron to generalize over, and the mixing in dense networks must come from interference under limited capacity. This is arbitrary packing in the essay's sense, and it confirms that it arises in this setting. It does not show how much polysemanticity in real networks this cause has, which is the essay's question.

The related-features experiment is a small step toward the essay's world. If neurons mix features because the features are related, dense neurons should line up with families when the features of a family tend to appear together. In Section III-G they did not. Dense neurons affected several families, more than with independent features, so in this toy the mixing is not confined to related features. Sparse training did produce neurons that serve one family, and these rarely reduced to a single feature. The essay reports that in a toy with correlated inputs and hidden neurons, related inputs grew onto shared neurons. Its setup differs from ours, so we do not claim that the two results agree or conflict.

D. Limitations

The study has clear limits. The setting is a synthetic toy with independent, sparse features of equal importance, and real data has none of these properties exactly. The results rest on three to five seeds per setting, and the differences between arms are small at p = 0.05, where error bars overlap. The learning-rate sweep covers only the dense network and GSE at 75% sparsity, at one width and one

feature probability, and the other comparisons use a single learning rate that is not the best one for the dense network, so the match in capacity between GSE at 50% and the dense network at 128 neurons has not been checked at other learning rates. Our GSE arm is an independent reimplementation of the method of [20] for a two-layer network. It ranks connections within each matrix, because ranking across both matrices, as the original method does, lost most of the capacity advantage in this network (Section III-C), so our conclusions about GSE apply to the per-matrix variant. The summary measure depends on definitions, such as which neurons count as important, that we fixed in advance but could have chosen differently. The visualizations come from a single seed and show the most important neurons, not a random sample. The comparison is also narrow in the methods it covers: the only weight-sparse method with rewiring that we tested is GSE, together with its fixed-mask counterpart, and the only activation-sparse method is plain top-k. Whether other prune-and-grow methods such as SET [17] and RigL [19] would fall on the same capacity and cleanliness curve is untested, so the conclusion that sparsity itself buys cleanliness while rewiring buys capacity should be read as a statement about the variants we tried. The experiment that starts with a trained network uses one pruning rule (magnitude within each matrix) and one fine-tuning budget (10,000 steps), so longer fine-tuning or other pruning rules might clean the neurons more. The experiment that retrains networks on the mask found by GSE uses 64 neurons and five seeds, and we did not test its differences for significance. Finally, our test of related features uses one family structure, 64 neurons and three seeds, and the toy still builds interference in, so the study cannot say how much of the polysemanticity in real networks comes from superposition rather than from recursive generalization [10].

E. Directions for future work

Several extensions follow directly. The first is the matched-capacity dense control described above. The second is to relax the assumptions of the toy: features that are correlated or organized in a hierarchy, features of unequal importance as in [4], and tasks other than the absolute value. Section III-G begins this investigation for related features, and it should be extended to more widths and seeds, to families of unequal size and to nested families, which would test recursive generalization directly against the interference account [10]. The third is to ask which weight matrix matters by constraining W1 and W2 separately, which would also test the counting-versus-optimization interpretation of Section IV-A. That a global ranking starves W1 suggests that the budget given to each matrix matters. The fourth is to repeat the comparison with other weight-sparse methods, such as SET [17] and RigL [19], and with a global ranking in deeper networks, to show whether the single curve is a property of sparsity itself or only of the variants we tried. The fifth is to combine the two approaches to interpretability by training sparse autoencoders on the activations of dense and of sparsely trained networks and comparing the quality of the features that they recover. The sixth, and the most important for relevance, is to move to small transformers and then to language models trained with sparse weights, evaluating them with sparse-autoencoder-based measures and with automated interpretation of neurons. There the central question is whether the capacity cost seen here is acceptable, since capacity is what large models need most, and whether sparsity can be applied to selected components, such as the feed-forward layers, where polysemantic neurons are most common. Finally, a theory of the capacity of sparse networks, relating the number of features that can be represented to sparsity and the feature probability, would turn the empirical trade-off curve of Fig. 3 into a prediction.

V. Conclusion

We asked whether training a network with sparse weights from the start makes its neurons easier to read. We also asked what this costs in capacity. In a toy model of superposition with known features,

always-sparse training with GSE made each neuron affect far fewer features when features were rare. At moderate sparsity it did so while keeping most of the capacity of the dense network, and in the wider network it kept all of it. The cleanliness of its neurons was stable across learning rates, whereas that of the dense network was not. The cost is capacity, which grows with sparsity and shrinks as the network gets wider. A fixed random mask gives even cleaner neurons but far fewer features, so the readability benefit comes from sparsity itself, and rewiring mainly recovers capacity. The effect weakens as features become denser, and ranking connections across both weight matrices removed most of the capacity benefit in this small network. Pruning a trained dense network kept the capacity but not the cleanliness, so the route to a sparse network matters. Retraining on the connectivity pattern that GSE found recovered much of its capacity when features were rare but gave less clean neurons than GSE. When features come in related families, dense neurons mixed several families, and sparse training made neurons serve one family but rarely a single feature. These findings come from a synthetic setting with a small number of seeds. Where sparse networks have lower capacity, a dense network of matched capacity is still missing, so we cannot yet say whether sparsity does more than lower capacity there. The results justify testing sparse training in small transformers, where the central question is whether the capacity cost is acceptable for the gain in readability.

Reproducibility

All experiments use one PyTorch script that trains every arm, computes the metrics and produces the figures, and runs on a single consumer GPU. Runs are resumable, and each run is stored as a separate file. A second script, which starts from trained dense networks, a third, which generates features that come in related families, and a fourth, which retrains networks on the connectivity patterns found by GSE, use the same model, metrics and GSE code. The code, the summary tables for every experiment and the figure scripts are released with the paper at https://github.com/narayan1966/sparsity-interpretability-toy.

References

[1] C. Olah, A. Mordvintsev, and L. Schubert, “Feature visualization,” *Distill*, Nov. 2017, doi: 10.23915/distill.00007.

[2] C. Olah, A. Satyanarayan, I. Johnson, S. Carter, L. Schubert, K. Ye, and A. Mordvintsev, “The building blocks of interpretability,” *Distill*, Mar. 2018, doi: 10.23915/distill.00010.

[3] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter, “Zoom in: An introduction to circuits,” *Distill*, Mar. 2020, doi: 10.23915/distill.00024.001.

[4] N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen, R. Grosse, S. McCandlish, J. Kaplan, D. Amodei, M. Wattenberg, and C. Olah, “Toy models of superposition,” *arXiv preprint arXiv:2209.10652*, 2022, doi: 10.48550/arXiv.2209.10652.

[5] D. O. Hebb, *The Organization of Behavior: A Neuropsychological Theory*. New York, NY, USA: Wiley, 1949.

[6] G. Buzsáki, “Neural syntax: Cell assemblies, synapsembles, and readers,” *Neuron*, vol. 68, no. 3, pp. 362–385, Nov. 2010, doi: 10.1016/j.neuron.2010.09.023.

[7] M. Rigotti, O. Barak, M. R. Warden, X.-J. Wang, N. D. Daw, E. K. Miller, and S. Fusi, “The importance of mixed selectivity in complex cognitive tasks,” *Nature*, vol. 497, no. 7451, pp. 585–590, May 2013, doi: 10.1038/nature12160.

[8] A. Scherlis, K. Sachan, A. S. Jermyn, J. Benton, and B. Shlegeris, “Polysemanticity and capacity in neural networks,” *arXiv preprint arXiv:2210.01892*, 2022, doi: 10.48550/arXiv.2210.01892.

[9] T. Henighan, S. Carter, T. Hume, N. Elhage, R. Lasenby, S. Fort, N. Schiefer, and C. Olah, “Superposition, memorization, and double descent,” *Transformer Circuits Thread*, Jan. 5, 2023. [Online]. Available: https://transformer-circuits.pub/2023/toy-double-descent/

[10] N. Cammarata, “Neural networks are already clean,” *Goodfire*, 2026. [Online]. Available: https://already-clean.goodfire.com/

[11] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. L. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah, “Towards monosemanticity: Decomposing language models with dictionary learning,” *Transformer Circuits Thread*, Oct. 4, 2023. [Online]. Available: https://transformer-circuits.pub/2023/monosemantic-features/

[12] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, “Sparse autoencoders find highly interpretable features in language models,” *arXiv preprint arXiv:2309.08600*, 2023, doi: 10.48550/arXiv.2309.08600.

[13] A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, A. Pearce, C. Citro, E. Ameisen, A. Jones, H. Cunningham, N. L. Turner, C. McDougall, M. MacDiarmid, A. Tamkin, E. Durmus, T. Hume, F. Mosconi, C. D. Freeman, T. R. Sumers, E. Rees, J. Batson, A. Jermyn, S. Carter, C. Olah, and T. Henighan, “Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet,” *Transformer Circuits Thread*, May 21, 2024. [Online]. Available: https://transformer-circuits.pub/2024/scaling-monosemanticity/

[14] L. Gao, T. Dupré la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu, “Scaling and evaluating sparse autoencoders,” *arXiv preprint arXiv:2406.04093*, 2024, doi: 10.48550/arXiv.2406.04093.

[15] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[16] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, “Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks,” *J. Mach. Learn. Res.*, vol. 22, no. 241, pp. 1–124, 2021.

[17] D. C. Mocanu, E. Mocanu, P. Stone, P. H. Nguyen, M. Gibescu, and A. Liotta, “Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science,” *Nat. Commun.*, vol. 9, Art. no. 2383, 2018, doi: 10.1038/s41467-018-04316-3.

[18] T. Dettmers and L. Zettlemoyer, “Sparse networks from scratch: Faster training without losing performance,” *arXiv preprint arXiv:1907.04840*, 2019, doi: 10.48550/arXiv.1907.04840.

[19] U. Evci, T. Gale, J. Menick, P. S. Castro, and E. Elsen, “Rigging the lottery: Making all tickets winners,” in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, Proc. Mach. Learn. Res., vol. 119, 2020, pp. 2943–2952.

[20] M. Heddes, N. Srinivasa, T. Givargis, and A. Nicolau, “Always-sparse training by growing connections with guided stochastic exploration,” in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2025, doi: 10.1109/IJCNN64981.2025.11229016.

[21] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. 13th Int. Conf. Artif. Intell. Statist. (AISTATS)*, Proc. Mach. Learn. Res., vol. 9, 2010, pp. 249–256.

[22] L. Gao, A. Rajaram, J. Coxon, S. V. Govande, B. Baker, and D. P. Mossing, “Weight-sparse transformers have interpretable circuits,” in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, Proc. Mach. Learn. Res., vol. 306, 2026, pp. 33483–33513.